closy



Not ML engineers. Cybersecurity engineers.

closy

AI security

An AI agent is an untrusted system

It reads text written by others and acts on someone's behalf. Security asks three questions of it.

Integrity

It says only what is true.

No invented number, no fact without a source.

Confidentiality

It never leaks.

What it must not reveal stays out of every message.

Control

Input never becomes an order.

What others write is data to read, not instructions to follow.

closy

AI security · Our agent

Threats to a buyer agent, and what covers them

Threat	Status	In our code
Prompt injection: a supplier e-mail tries to steer the agent	Covered by design	The reader treats the e-mail as untrusted data; a signal is kept only if its quote is word for word on the cited line. Offers are set by code, not by the model. `reader.py`
Invented facts or figures	Covered	The guard drops any sentence whose ID is unknown or whose figure does not match its source. `guard.py`
Leaking our alternative, budget or urgency	Covered	The guard blocks our alternative, its price, budget and urgency words before sending. `guard.py`
Crash or stall during the final	Covered and tested	Runner fix: 5 crashes in 45 parallel games → 0 in 40; 0 on 21 cases with the 60 s clock (local simulator). `R2`, `R3`
Membership inference, training-data extraction	Not applicable	We train no model on company data; the optional LLM is only called at run time. `no training code`
Adversarial testing of these defences	Not done	We did not run attacks against the agent. The defences above are design and code, not red-team results.

Agent `hollerau/pegasus` at `4fabd57`; vault R2, R3. "Covered" means the code path exists. The guard has a unit test (`tests/test_pegasus.py`); the reader's quote rule has none. Neither is an attack result.

closy

Why AI security matters for a buyer

€

A wrong figure costs money

The buyer signs on the agent's numbers.

0

A leak weakens every offer

A supplier who knows our alternative or our urgency stops conceding.

0

A crashed case scores zero

In the final, ten games run at once; one failure costs the whole case.

closy

How we did it · Execution

Built to finish before the deadline

The runner manages concurrent negotiations and closes cases before the time cap.

01 · Run in parallel

10

concurrent cases

Each supplier thread has a worker. Shared decisions within a case use a lock.

02 · Watch the clock

Negotiate → close

When the cap is near, use a closing move. If agent closing fails, fall back to the kit's closing logic.

03 · Rehearse the execution

21 / 21

cases completed

0 crashes / timeouts

55 ms median decision

880 ms slowest decision

Local simulator rehearsal, 11 Oct 09:40, main 4fabd57; 144 decision calls, parallel=10, 60-second cap. Execution evidence, not a benchmark of hidden-supplier deal quality. Rehearsal log ↗ · Runner ↗

closy

Our build approach · Experimentation

We tested ideas—and rejected the ones that failed

A new strategy needed evidence before it entered the final agent.

01 · Hypothesis

Predict the supplier's price floor

Use its first counter-offer and the case category to estimate how far it will move.

02 · Set the test

137

archived sessions · 8 cases

Leave one case out at a time. Set success criteria before computing the result.

03 · Compare with the criterion

Required error	Observed error
≤ 2%	8.6%

Error measured relative to the opening price. The other success criteria also failed.

04 · Decide

Rejected

All three R4 ideas failed their prewritten criteria. They did not change the final version.

Every offer is calculated. Every argument is sourced.
Every change must earn its place.

R4: offline experiments on 11 Oct; no arena sessions. The price-floor model, frozen-price policy and concession-curve model were rejected. Criteria and results ↗